



How do we address it?

- We could develop a different operational semantics?

This would making the scheduling of behaviours more complex

We still wouldn't have a design of BoC independent of the machinery that realises it

We may end up being too restrictive or permissive again

- Can we take inspiration from the way others have formulated these designs, namely weak memory models?

Can we construct an axiomatic model of concurrency that separates intrinsic and incidental causality?

An Axiomatic Concurrency Model

```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
    if(account.balance + amount > 0) {  
      account.balance += amount  
      val id = account.id()  
      when(log) { B log.record(id, amount) }  
    } } }
```

update(src, +100)

update(src, -100)

Candidate executions are tuples of $(\mathbb{E}, po, co)$

$\mathbb{E}$ is the set of program events

po is the program order representing the order in which a behaviour spawns subsequent behaviours

co is the cown order representing the order between one behaviour completing and subsequent behaviour running